```
      oidcUser.getPreferredUsername(),
      oidcUser.getGivenName(),
      oidcUser.getFamilyName(),
      List.of("employee", "customer")
    );
    return Mono.just(user);                    ❸
  }
}
```

❶ Injects an OidcUser object containing info about the currently authenticated user

❷ Builds a User object from relevant claims contained in OidcUser

❸ Wraps the User object in a reactive publisher, since Edge Service is a reactive application

Ensure that Keycloak and Redis are still running from the previous section, run the Edge Service application (`./gradlew bootRun`), open an incognito browser window, and navigate to http://localhost:9000/user. Spring Security will redirect you to Keycloak, which will prompt you to log in with a username and password. For example, authenticate as Bjorn (`bjorn/password`). After successfully authenticating, you will be redirected back to the `/user` endpoint. The result is the following:

```
{
  "username": "bjorn",
  "firstName": "Bjorn",
  "lastName": "Vinterberg",
  "roles": [
    "employee",
    "customer"
  ]
}
```

NOTE The `roles` list includes hardcoded values. In the next chapter, we'll change it to return the actual roles assigned to each user in Keycloak.

When you're done trying out the new endpoint, stop the application with Ctrl-C and the containers with `docker-compose down`.

Consider what happened when you tried to access the `/user` endpoint and got redirected to Keycloak. After successfully validating the user's